ECE 425-525: Course Project, Final Report

Project ID: P04

VLSI Design Agent

An interactive toolkit for RTL generation, testing, and improvement

To write: Team members' names and roles (agent and integration; EDA and evaluation; dashboard and delivery).

At a glance. A local LLM (gemma4:26b) writes Verilog and a self-checking testbench from a specification; Icarus Verilog and Yosys check it, and the agent revises from that feedback for up to 3 attempts. Hidden reference tests the agent never sees score every attempt. Over 200 paired runs (4 strategies, 10 circuits, seeds 1–5), revising fixed 10 one-shot failures and broke 4 correct designs. The main obstacle was the model's own testbench: on held-out circuits it rejected 54–81% of the designs that were in fact correct.

To write: Abstract (150 words): problem, what was built, how it was evaluated, the headline numbers above, and the main lesson (iteration repairs compile-level errors far more often than logic errors; the model's own tests are the bottleneck).

1. Introduction

To write: The proposal's "aha": Verilog that looks right can still describe the wrong circuit. Who the toolkit is for (students of digital design, VLSI and agentic AI). The question we test: does feeding EDA-tool feedback back to the model beat one-shot generation, judged by tests the agent cannot see? Contributions: the agent and its two workflows, the honest evaluation (hidden tests, paired scores, a held-out split declared in advance), the dashboard, and the containerised delivery. Outline of the report.

2. Related work

To write: Step 4 of the plan, 1 to 1.5 pages, every citation checked against the paper itself: LLM-for-RTL benchmarks (e.g. VerilogEval, RTLLM), feedback-driven agents (e.g. AutoChip), fine-tuned RTL models (e.g. RTLCoder, ChipNeMo), and what our study adds (paired one-shot vs. iterative scores on the same runs, the agent's own checks measured against an independent test, a declared held-out split).

3. System design

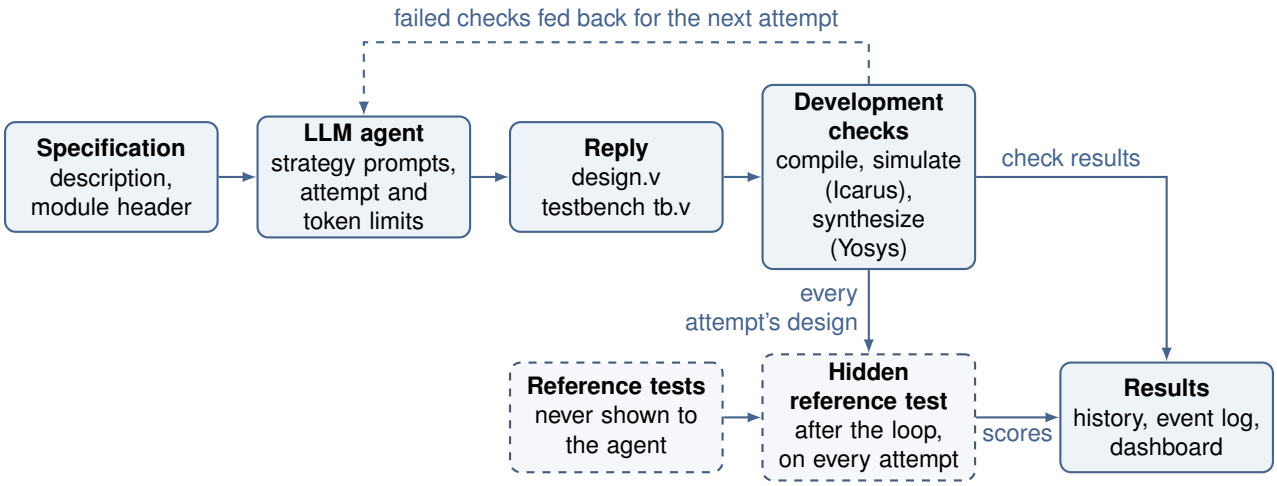


Figure 1: The agent loop. Only the development checks reach the model; the hidden reference test scores the designs after the loop ends.

To write: The loop (Figure 1): the `single` workflow (one reply holds design and testbench) and `spec_first` (the testbench is written from the specification first, in its own conversation, and its author reviews disputed checks). Limits: 3 attempts, 50,000 tokens. What the development checks report back. Every run keeps its event log, history and the prompts it used.

3.1 Keeping the evaluation honest

To write: Hidden reference tests are written by the team and stay outside the agent's view; each run replaces their result-line placeholder with a random token, so generated code cannot print a fake pass. Each attempt is scored after the loop, so one run gives a paired one-shot (attempt 1) and iterative (final) score. The tools run with timeouts and capped output. The 58 fixtures (20 correct, the rest deliberately faulty) check the tests themselves (`docs/acceptance-checks.md`).

3.2 Circuits

Circuit	Family	Split	Hidden-test checks	Fixtures correct / faulty
`adder4`	adder	Dev	512	2 / 3
`mux2`	mux	Dev	524	2 / 4
`counter4`	counter	Dev	249	2 / 4
`alu4`	alu	Dev	2,048	2 / 5
`seqdet_moore`	fsm	Dev	686	2 / 4
`adder8`	adder	Held-out	131,072	2 / 3
`mux4`	mux	Held-out	1,064	2 / 3
`updown8`	counter	Held-out	380	2 / 4
`alu8`	alu	Held-out	524,288	2 / 4
`seqdet_mealy`	fsm	Held-out	684	2 / 4

Table 1: The 10 circuits. Strategies were tuned on the dev circuits only; the held-out circuits were used once per strategy, with the runs declared in advance.

4. The toolkit

To write: What a student does with it, by dashboard section: (1) pick a circuit, strategy, mode and model; (2) watch the run live; (3) inspect every attempt: design, testbench, changes from the previous attempt, the hidden test's mismatches, waveforms, the feedback the model got, and editing the RTL to re-test it; (4) benchmark charts; (5) the strategy editor. Exports. Then delivery: Docker image with pinned tool versions, Compose with Ollama on the GPU, the image published by CI after its tests pass.

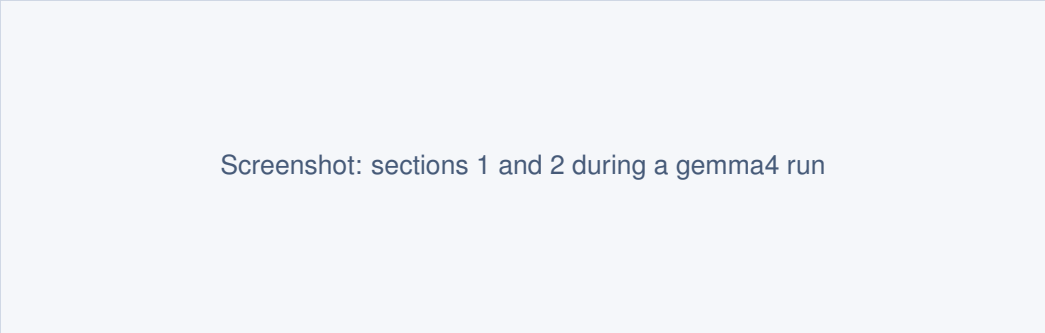


Figure 2: Running the agent: inputs and the live log.

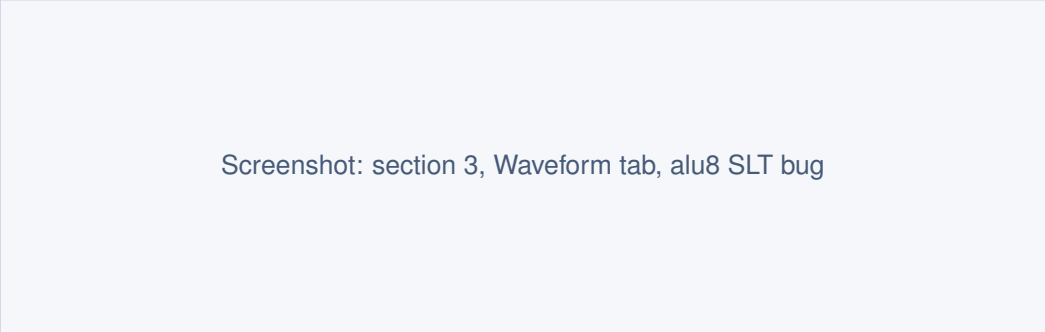


Figure 3: One attempt in the results view, with the waveform of a failing hidden test.

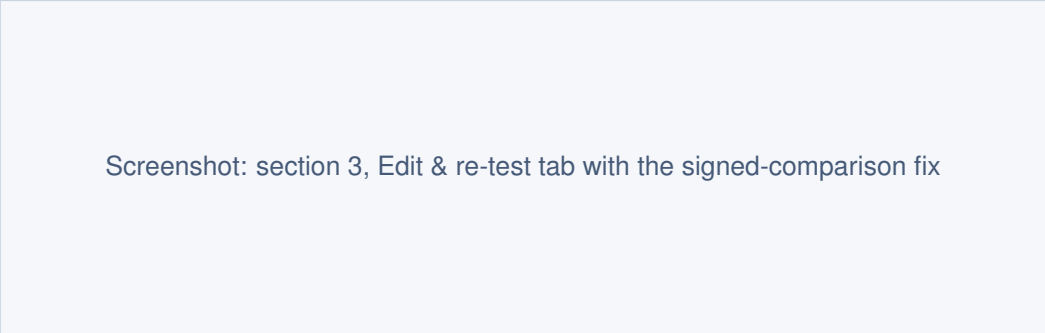


Figure 4: Editing an attempt's RTL and re-testing it.

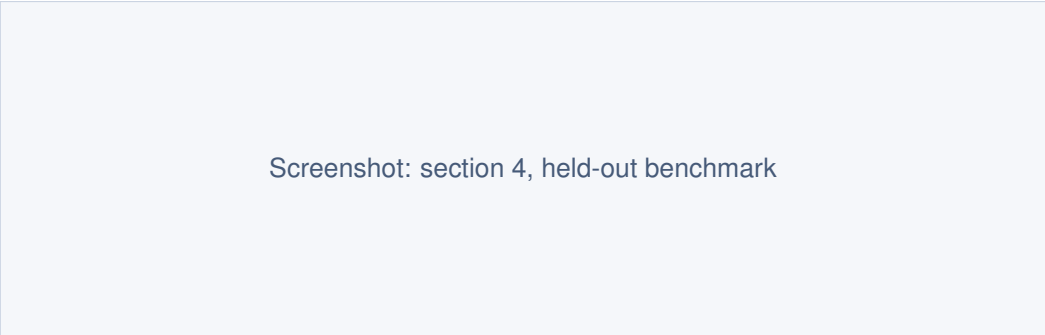


Figure 5: Benchmark charts.

5. Experimental method

To write: Model gemma4:26b through Ollama, temperature 0.2, fixed seeds 1–5 (the same seed reproduced a run exactly at this temperature). Up to 3 attempts and 50,000 tokens per run. Four strategies: `default` (the baseline, unchanged), `tb_timing`, `spec_first` v1 and v2. Every attempt scored, so one-shot and iterative come from the same runs (paired). Metrics as promised in the proposal: compilation, hidden-test pass rate, synthesis, retries, latency and tokens; cell counts only for passing designs under the same Yosys flow. Order of work: tune on dev, declare, run held-out once, no tuning after.

6. Results

Strategy	One-shot	Iterative	Fixed	Broken	Pass after 1 / 2 / 3	Own checks reject correct	Mean tokens	Mean time
Held-out circuits								
default	22/25	24/25	2	0	22 / 24 / 24	81%	10,501	23.7 s
tb_timing	25/25	25/25	0	0	25 / 25 / 25	54%	6,780	18.3 s
spec_first v1	20/25	21/25	2	1	20 / 21 / 21	65%	12,526	26.1 s
spec_first v2	20/25	23/25	3	0	20 / 23 / 23	59%	12,817	25.9 s
Dev circuits								
default	24/25	24/25	0	0	24 / 24 / 24	51%	7,296	16.6 s
tb_timing	23/25	22/25	0	1	23 / 22 / 22	36%	5,906	15.9 s
spec_first v1	23/25	24/25	1	0	23 / 24 / 24	21%	5,985	15.4 s
spec_first v2	23/25	23/25	2	2	23 / 24 / 23	32%	6,744	15.6 s

Table 2: One-shot vs. iterative, per strategy and split (25 runs each). Green: revising helped overall; red: it hurt. "Own checks reject correct": share of correct designs (by the hidden test) that the agent's own development checks failed.

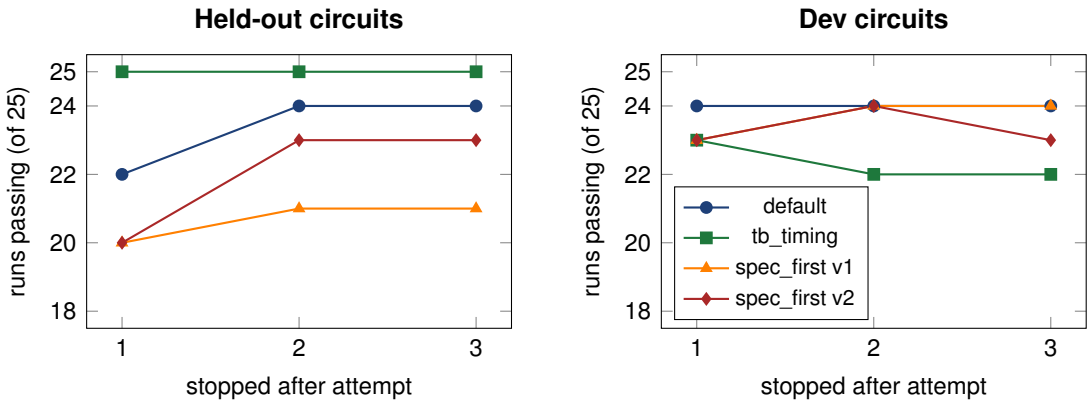


Figure 6: Hidden-test passes if each run had stopped after 1, 2 or 3 attempts (attempt 1 is the one-shot result). The vertical axis starts at 17.5.

Circuit	default	tb_timing	spec_first v1	spec_first v2	Median cells
`adder8`	5	5	5	5	52
`mux4`	3 → 5	5	5	5	15
`updown8`	5	5	5 → 4	5	47
`alu8`	4	5	4	4	256
`seqdet_mealy`	5	5	1 → 3	1 → 4	9

Table 3: Held-out circuits: passing runs out of 5, one-shot → iterative, and the median cell count of passing final designs.

Circuit	default	tb_timing	spec_first v1	spec_first v2	Median cells
`adder4`	5	5	5	5	23
`mux2`	5	5	5	5	8
`counter4`	5	5	5	5 → 3	10
`alu4`	4	4 → 3	3 → 4	3 → 5	134
`seqdet_moore`	5	4	5	5	18

Table 4: Dev circuits, as in Table 3.

6.1 How reliable were the agent's own checks?

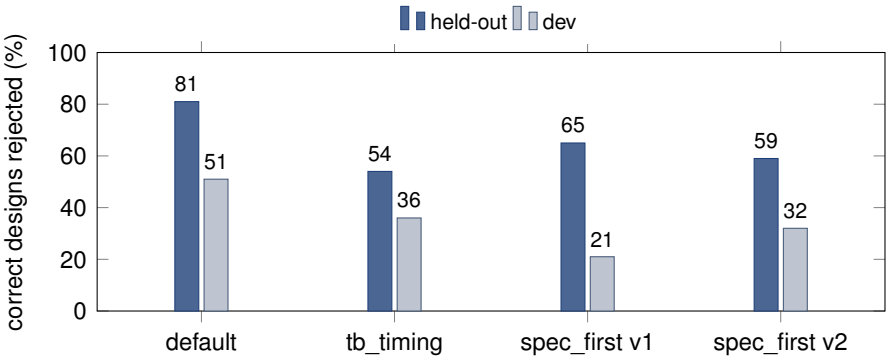


Figure 7: Share of correct designs (passing the hidden test) that the agent's own development checks rejected, over all attempts.

	Correct designs rejected		Wrong designs accepted	
Strategy	count	share	count	share
Held-out circuits				
default	47 of 58	81%	0 of 5	0%
tb_timing	22 of 41	54%	none	–
spec_first v1	24 of 37	65%	2 of 11	18%
spec_first v2	22 of 37	59%	1 of 8	12%
Dev circuits				
default	20 of 39	51%	0 of 3	0%
tb_timing	12 of 33	36%	2 of 6	33%
spec_first v1	6 of 29	21%	1 of 2	50%
spec_first v2	10 of 31	32%	1 of 5	20%

Table 5: The agent's own checks against the hidden test, over all scored attempts.

		Right call		Wrong call	
Version	Reviews	revised, design correct	kept, design wrong	kept, design correct	revised, design wrong
Held-out circuits					
spec_first v1	28	15	1	9	3
spec_first v2	26	6	1	18	1
Dev circuits					
spec_first v1	12	10	0	0	2
spec_first v2	14	1	3	10	0

Table 6: `spec_first`: testbench reviews after a design failed some checks, judged by the hidden test of the design under review.

To write: Walk the reader through the tables: revising fixed 10 and broke 4 of 200 paired runs; on held-out, `default` went 22/25 → 24/25 and `spec_first` v2 20/25 → 23/25; over both splits the strategies score 45–48/50 and their order flips between splits. Own checks rejected 54–81% of correct held-out designs (21–51% on dev). Cost: iterative runs used several times the one-shot tokens.

7. Discussion

To write: What iteration fixes: errors the tools name (compile errors, e.g. assigning a non-reg output in an always block). What it does not fix: specification-level logic bugs (the unsigned SLT comparison survived every revision). Why: the model's testbench shares the design's misconception, has timing bugs, or rejects correct designs, so the feedback is often wrong (the proposal's "biggest roadblock"). The `spec_first` result: separating the testbench author reduced false rejections on dev but not on held-out. Strategy differences are within run-to-run noise at 25 runs per cell. Examples from the benchmark report (docs/benchmark-report.html).

8. Limitations and threats to validity

To write: Five seeds per setting; one model; small circuits; reference tests written by the team (checked by fixtures, but not proofs); cell counts are not power, area or timing; GPU nondeterminism; RAG and LoRA fine-tuning (stretch goals) not attempted.

9. Reproducibility

To write: The repository (github.com/Harrydhara/VLSI-Design-Agentic-Framework), the image (ghcr.io/harrydhara/vlsi-design-agentic-framework) with pinned versions (Icarus Verilog 12, Yosys 0.57+260 built from the benchmark commit, Ollama 0.31.2, exact Python packages). Commands: rerun a benchmark (`vlsi-agent bench`), regenerate these tables (`generate.py`), and the HTML report. All runs behind this report are committed under `runs/bench/`.

10. Deliverables against the proposal

To write: One table: each proposal promise (MVP, dashboard features, evaluation, final deliverables) and each member's completion check from the division of work, with the evidence (docs/acceptance-checks.md: all three checks pass) and any gaps (RAG and LoRA not attempted; an FPGA board was optional and not used).

11. Conclusion and future work

To write: The answer to the research question in two sentences. Future work: better testbench feedback (a harness that samples at the right moment), RAG over Verilog examples, LoRA fine-tuning with separate data, multi-agent review, larger circuits (a FIFO pair), delivering the last compiling design.

A. A worked example: the unsigned SLT bug

To write: One run end to end (runs/bench/heldout-seeds1-5/runs/alu8/default/seed4): the specification, the design's 9-bit sign-extended but unsigned comparison, the own testbench's verdict, the feedback, three unchanged attempts, the hidden test's mismatches and waveform, and the one-word manual fix that passes 524,288/524,288 (runs/manual/20261020-141335-alu8-edit) while the model's testbench still reports 0/22.

B. The baseline prompts

The `default` strategy, unchanged throughout the project.

system.txt

```
You are a careful digital design engineer. You write synthesizable Verilog-2005 RTL and a self-checking
↪  testbench for it.

Rules for the design:
- Use exactly the module header you are given: the same module name, port names, directions and widths.
- Write synthesizable RTL only: no # delays, no initial blocks, and no $display or other system tasks in the
↪  design.
- Combinational logic: use assign, or always @(*) that assigns every output in every branch, so no latches are
↪  inferred.
- Sequential logic: use always @(posedge clk ...) with nonblocking assignments (<=).
- Helper modules are allowed; put them in the same code block as the design.

Rules for the testbench:
- Name the testbench module exactly tb, with no ports, and instantiate the design inside it.
- Apply inputs and compare every output with the value the specification requires. Cover corner cases.
- If the design has a clock, generate it with: always #5 clk = ~clk;
- Count the checks. For every failed check, print a line that starts with FAIL: and shows the inputs, the
↪  expected value and the actual value.
- At the end, print exactly one summary line and stop:
    $display("RESULT: %0d/%0d passed", passed, total);
    $finish;

Reply format: exactly two fenced code blocks and nothing else that is code. First a ```verilog block with the
↪  design, then a ```verilog block with the testbench.
```

task.txt

```
Design the following circuit.

{{description}}

Use exactly this module header:
{{interface}}

Write the design (top module {{top}}) and a self-checking testbench module named tb.
```

feedback.txt

```
Attempt {{attempt}} of {{max_attempts}} did not pass the development checks:

{{feedback}}

Fix the problems. If a check failed because the testbench itself is wrong, fix the testbench; otherwise fix the
↪  design. Reply with both complete code blocks again: the design first, then the testbench.
```